F.CSM302 ҮЙЛДЛИЙН СИСТЕМ · БИЕ ДААЛТ 1

Scheduling Algorithms

CPU scheduling алгоритмуудын C хэрэгжилт, Gantt chart бүхий интерактив симулятор, туршилтын тайлан

Хуваарилалтын код	IV-2-8 (IV өдөр · 2-р цаг · 8-р сэдэв)
Төсөл	Scheduling Algorithms — Operating System Concepts (10th ed.), 5-р бүлэг, Programming Projects
UI дүрслэл	Gantt chart (waiting, turnaround, response time-тай)
Багийн гишүүд	**[Нэр, оюутны код — 1 хүн тутамд мөр нэмнэ]**
GitHub	**[repository холбоос]**
Release / Tag	**[9-р долоо хоногийн tag, жишээ нь v1.0]**
Огноо	2026 оны 10-р сар

Хураангуй

Энэхүү ажлаар сурах бичгийн 5-р бүлгийн "Scheduling Algorithms" Programming Project-ийг гүйцэтгэв. FCFS, SJF, Priority, Round-Robin, Priority with Round-Robin гэсэн таван алгоритмыг **C хэл** дээр, номын бүтцийн дагуу (*driver.c*, *CPU.c*-ийн *run()*, холбоост жагсаалт, *Makefile*) хэрэгжүүлсэн. Нэмэлт даалгаваруудын дагуу task id-г *__sync_fetch_and_add*-аар атомаар олгож, алгоритм бүрийн дундаж waiting, turnaround, response time-ийг тооцдог.

Багшийн оноосон **Gantt chart** дүрслэлийг хөтөч дээр ажилладаг интерактив симуляторoор хэрэгжүүлсэн. Хэрэглэгч arrival, burst, priority, quantum болон алгоритмыг өөрчилж ажиллуулж, цагийн алхмаар Gantt chart, ready queue, үзүүлэлтүүдийг ажиглана. C хэрэгжилт ба симулятор нь **хоёр тусдаа хэрэгжилт** бөгөөд бие биенийхээ үр дүнг баталгаажуулдаг.

Номын жишээ (T1…T5) дээр дундаж waiting time: FCFS 44.00, SJF 30.00, Priority 33.00, RR 57.00, Priority+RR 35.00 ms. Автомат тест 18 шалгалтаас 18-г нь давсан.

1. Удиртгал

CPU scheduler нь ready queue дахь процессуудаас аль нэгийг сонгож CPU олгодог. Сонголтын дүрэм (алгоритм) нь системийн waiting, turnaround, response time зэрэг үзүүлэлтэд шууд нөлөөлдөг. Ажлын зорилго нь алгоритмуудыг өөрсдөө программчилж, ижил өгөгдөл дээр харьцуулан ажиглах, үр дүнг нүдэнд харагдахуйц Gantt chart-аар батлах юм.

Хамрах хүрээ: бүх task t = 0 үед ирнэ (номын нөхцөл), priority 1–10 (их тоо = өндөр), quantum анхны утга 10 ms. Симулятор нэмэлтээр arrival time болон preemptive хувилбарууд (SRTF, preemptive Priority)-ыг дэмждэг боловч C хувилбар нь номын нөхцөлөөр хязгаарлагдана.

2. Сурах бичгийн шаардлага

Номын P-29…P-31 хуудсанд заасан үндсэн шаардлага:

- Таван алгоритм: FCFS, SJF, Priority, Round-Robin, Priority with Round-Robin (тэнцүү priority-той task-ууд RR-аар ээлжилнэ).
- Priority 1–10, их тоо = өндөр. RR-ийн time quantum = 10 ms.
- Оролт: *schedule.txt*, мөр бүр *[task name], [priority], [CPU burst]* хэлбэртэй (жишээ: T1, 4, 20). Бүх task нэгэн зэрэг ирнэ.
- C эсвэл Java хэрэгжилт; *driver* нь task-уудыг жагсаалтад оруулж *schedule()*-ийг дуудна; сонгосон task-ийг *CPU.c*-ийн *run()* гүйцэтгэнэ; алгоритмыг *Makefile*-аар сонгоно (*make fcfs*, *./fcfs schedule.txt*).
- Нэмэлт 1: task id-г SMP орчинд race condition-гүй олгох (атомар increment).
- Нэмэлт 2: алгоритм бүрийн дундаж turnaround, waiting, response time-ийг тооцох.

Эдгээр шаардлага бүгд хэрэгжсэн (8-р бүлгийн хүснэгт ба Хавсралт B-г үзнэ үү).

3. Онолын үндэс

Үзүүлэлтүүд. Бүх task t = 0 үед ирдэг тул: *Turnaround = Completion − Arrival*, *Waiting = Turnaround − Burst*, *Response = Эхний удаа CPU авсан цаг − Arrival*. Arrival-тай ерөнхий тохиолдолд ч томьёо ижил.

FCFS — ирсэн дарааллаар. Хэрэгжүүлэхэд хамгийн хялбар боловч урт task урд байвал богино task-ууд удаан хүлээдэг (convoy effect).

SJF — дараагийн CPU burst хамгийн богиныг сонгоно. Бүх task нэгэн зэрэг ирэх үед дундаж waiting time-ийг **хамгийн бага** болгодог (оновчтой).

Priority — хамгийн өндөр priority-тойг сонгоно. Бага priority-той task нь байнга хойш тавигдаж **starvation** үүсэж болно.

Round-Robin — task бүр ээлжлэн ≤ quantum хугацаагаар ажиллаж, дуусаагүй бол дарааллын сүүлд орно. Response time сайн, харин context switch их, дундаж turnaround өндөр байдаг.

Priority + RR — priority-оор эхлээд сонгож, тэнцүү priority-той task-ууд дотроо RR ээлжилнэ.

4. Системийн зохиомж

C хэрэгжилт нь номын загварыг дагана. Алгоритмаас хамаарахгүй хэсэг (*driver.c, CPU.c, list.c, tasks.c*) болон алгоритм тус бүрийн нэг файл (*fcfs.c, sjf.c, priority.c, rr.c, priority_rr.c*) тус бүр *add()*, *pickNextTask()*, *schedule()* гэсэн ижил интерфэйс хэрэгжүүлнэ. *Makefile* нь алгоритмын файлыг нийтлэг хэсэгтэй холбож тусдаа executable үүсгэнэ.

Файл	Үүрэг
driver.c	schedule.txt уншиж шалгах (priority 1–10, burst ≥ 1, формат), quantum аргумент, add() → schedule() → report()
list.c / list.h	Холбоост жагсаалт: insert (сүүлд нэмнэ), delete
CPU.c	run(task, slice): "Running task = ..." хэвлэж, цаг ахиулж, start/finish тэмдэглэнэ
tasks.c	new_task(): атомар tid, бүх task-ийн бүртгэл; report(): waiting/turnaround/response ба дундаж
fcfs.c ... priority_rr.c	pickNextTask() болон schedule()-ийн алгоритм тус бүрийн хэрэгжилт
src/web/index.html	Симулятор: оролтын хүснэгт → simulate() → Gantt, метрик, pseudo-code, алхамчилсан тоглуулалт

Өгөгдлийн урсгал: *schedule.txt → driver.c (шалгалт) → add() → алгоритмын жагсаалт → schedule() → pickNextTask() → run() → report()*. Симулятор дээр: *оролтын хүснэгт → readInput() (шалгалт) → simulate() → log[t] (цаг тутмын төлөв) → Gantt + код тодруулалт*.

5. Хэрэгжилт

Хэл, орчин. C (gnu11, gcc), Make; симулятор нь гадны сангүй цэвэр HTML/CSS/JavaScript нэг файл.

Tie-breaking. SJF ба Priority-д тэнцүү утгатай бол файлд эхэнд байсан task-ийг сонгоно (*pickNextTask()* дэх хатуу харьцуулалт). Priority+RR-д жагсаалтын дараалал нь ээлжийг тодорхойлно.

Round-Robin. Task бүр *min(quantum, remaining)* ажиллана; дуусаагүй бол жагсаалтын сүүлд дахин оруулна:

```
#include <stdlib.h>
#include "schedulers.h"
#include "list.h"
#include "CPU.h"
static struct node *head = NULL;
void add(char *name, int priority, int burst) { insert(&head, new_task(name, priority, burst)); }
Task *pickNextTask(void) { return head ? head->task : NULL; }
void schedule(void) {
    Task *t;
    while ((t = pickNextTask())) {
```

```
        int slice = t->remaining < quantum ? t->remaining : quantum;
        run(t, slice);
        delete(&head, t);
        if (t->remaining > 0) insert(&head, t);
    }
}
```

Атомар tid (Нэмэлт 1). Task id-г *__sync_fetch_and_add(&next;_tid, 1)*-ээр олгосон тул олон CPU-д зэрэг дуудагдсан ч давхардахгүй:

```
static int next_tid = 1;
Task *new_task(char *name, int priority, int burst) {
    t->name = strdup(name);
    t->tid = __sync_fetch_and_add(&next_tid, 1);   /* atomic increment: no race on tid */
    printf("\n%-8s %4s %4s %6s %9s %11s %9s\n", "Task", "tid", "Pri", "Burst", "Waiting", "Turnaround", "Response");
        printf("%-8s %4d %4d %6d %9d %11d %9d\n", t->name, t->tid, t->priority, t->burst, wait, tat, resp);
```

Метрик (Нэмэлт 2). *report()* нь task бүрийн waiting, turnaround, response болон 3 дундажийг хэвлэнэ (8-р бүлэг).

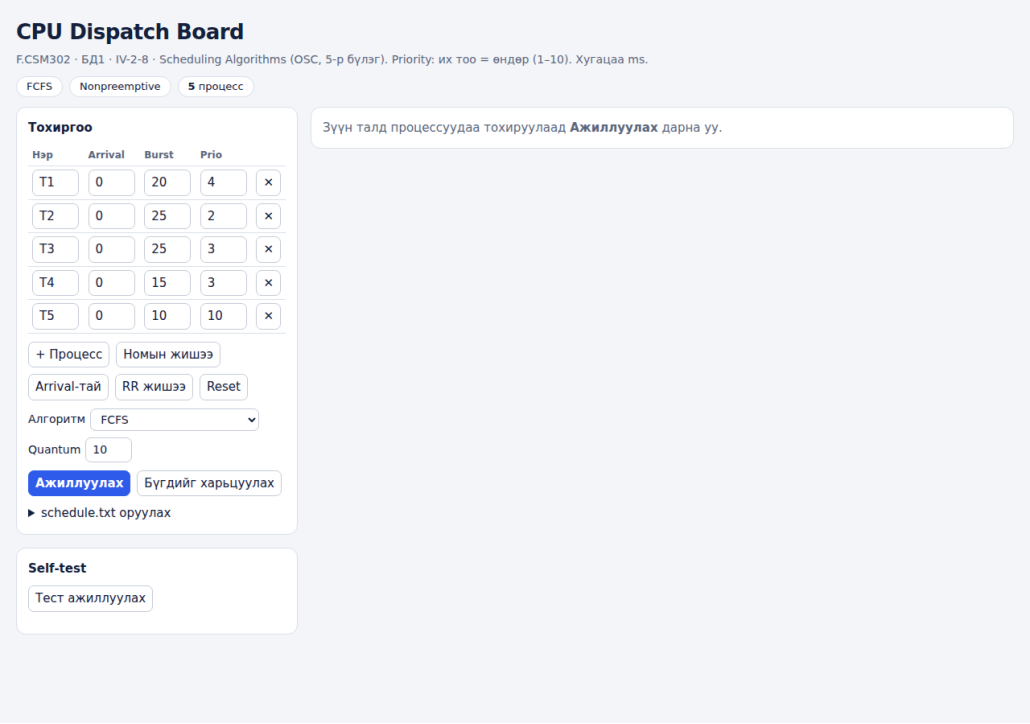
Хоёр хэрэгжилт. Симулятор нь C-ээс тусдаа логиктой (1 ms-ийн алхамтай тоглуулалт). Хоёрыг ижил өгөгдлөөр тулгаж шалгасан нь алдааг илрүүлэх нэмэлт баталгаа болсон. Нэмэлтээр UI нь C-ийн бодит гаралтыг уншдаг (*parseC()*): мөр бүрийг шалгаж (хугацаа тасралтгүй, task бүрийн нийт ажилласан хугацаа = burst), waiting / turnaround / response-ийг дахин тооцно.

6. Туршилт ба UI дүрслэл

Симулятор (*src/web/index.html*) дээр дараах зүйлийг өөрчилж ажиглана:

- **Процессын хүснэгт:** нэр, arrival, burst, priority — засах, нэмэх, устгах; "schedule.txt оруулах" хэсэгт номын форматаар буулгах.
- **Алгоритм ба quantum:** 7 алгоритм (таван үндсэн + SRTF + preemptive Priority); quantum нь RR-д л идэвхтэй.
- **Gantt chart:** дээд мөр = CPU (процесс бүр өөр өнгөтэй, idle = судалтай); процесс бүр тусдаа мөртэй — тод хэсэг нь CPU дээр ажилласан, бүдэг шугам нь ready queue-д хүлээсэн үе (waiting). Улаан босоо шугам = одоогийн хугацаа.
- **Алхамчилсан тоглуулалт:** slider, ◀ ▶, □ □, "Тоглуулах". Алхам бүрд pseudo-code-ийн биелж буй мөр тодорч, *PICK / RUN / PREEMPT / DONE* тайлбар, context switch, дууссан тоо, ready queue харагдана.
- **Метрикийн хүснэгт** ба **"Бүгдийг харьцуулах"** — 7 алгоритмын дунджийг зэрэгцүүлнэ.
- **С-ийн гаралт оруулах:** С програмын гаралтыг (*Running task … (t = a..b)* мөрүүд) буулгахад UI нь тэр **бодит гаралтаас** Gantt ба метрикийг зурж, Code Viewer-т С-ийн мөрүүдийг тодруулна. Ингэснээр UI нь С-ийн жинхэнэ ажиллагаатай шууд холбогдоно (8-р бүлгийн Зураг 9).

Анхаарах: симуляторын горимд Code Viewer-т харагдах код нь алгоритмын гогцооны **pseudo-code** бөгөөд бодит *.c* файл биш (гол ялгаа нь *pickNextTask* ба preempt нөхцөл). "С-ийн гаралт оруулах" горимд Code Viewer нь С програмын бодит гаралтын мөрүүдийг харуулна. Бодит С код нь *src/c/*-д байна.



Зураг 1. Симулятор — анхны төлөв (номын жишээ ачаалагдсан).

7. Туршилтын арга

Орчин 1 — баг гараар шалгасан (macOS): macOS **[macOS хувилбар]**, Terminal (zsh), *gcc* **[gcc --version-ийн гаралт, Mac дээр Apple clang]**, Safari. *make* анхааруулгагүй хөрвүүлэгдэж, *make test* 18/18 давсан; С демо ба вэб демог энэ орчинд бичлэгт буулгасан.

Орчин 2 — автомат шалгалт (Linux): gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0; v22.22.2; Linux 6.18.44-fc-v153 x86_64; Chromium 141.0.7390.37. Симулятор Chromium-д Playwright-аар автоматаар туршигдсан (зарим screenshot ингэж авагдсан).

Өгөгдөл: (1) номын жишээ *tests/schedule.txt* — T1,4,20; T2,2,25; T3,3,25; T4,3,15; T5,10,10. (2) *tests/schedule_ties.txt* — тэнцүү priority-той гурван task (A,B,C priority 5) ба нэг бага priority-той D, ээлж болон tie-breaking шалгана. (3) Алдаатай оролтууд: *bad_priority, bad_burst, bad_format, bad_empty*.

Хүлээгдсэн үр дүн: *tests/expected.txt* — C-ээс бие даасан JavaScript симуляторoор тооцсон дундаж waiting/turnaround/response. *make test* нь C-ийн гаралтыг эдгээртэй харьцуулна. Мөн гараар шалгах боломжтой: FCFS-д waiting = 0, 20, 45, 70, 85 → дундаж 44.

Хамрах шалгалт (18): 10 алгоритм-өгөгдлийн хослол (quantum 10 ба 4), 4 алдаатай файл, 3 буруу quantum (0, −3, abc), байхгүй файл. Нэмэлтээр симуляторын self-test 12 шалгалт (C-гаралт уншигчийн 3 шалгалт орно).

8. Үр дүн

Номын жишээ дээрх дундаж үзүүлэлтүүд (ms), C хэрэгжилтийн гаралтаас:

Алгоритм	Avg waiting	Avg turnaround	Avg response
FCFS	44.00	63.00	44.00
SJF	30.00	49.00	30.00
Priority	33.00	52.00	33.00
Round Robin (q=10)	57.00	76.00	20.00
Priority + RR (q=10)	35.00	54.00	30.00

Task бүрийн waiting time (ms):

Task (priority, burst)	FCFS	SJF	Priority	Round Robin (q=10)	Priority + RR (q=10)
T1 (4, 20)	0	25	10	40	10
T2 (2, 25)	20	45	70	65	70
T3 (3, 25)	45	70	30	70	45
T4 (3, 15)	70	10	55	70	50
T5 (10, 10)	85	0	0	40	0

C хэрэгжилтийн бодит гаралт (FCFS, *./fcfs tests/schedule.txt*):

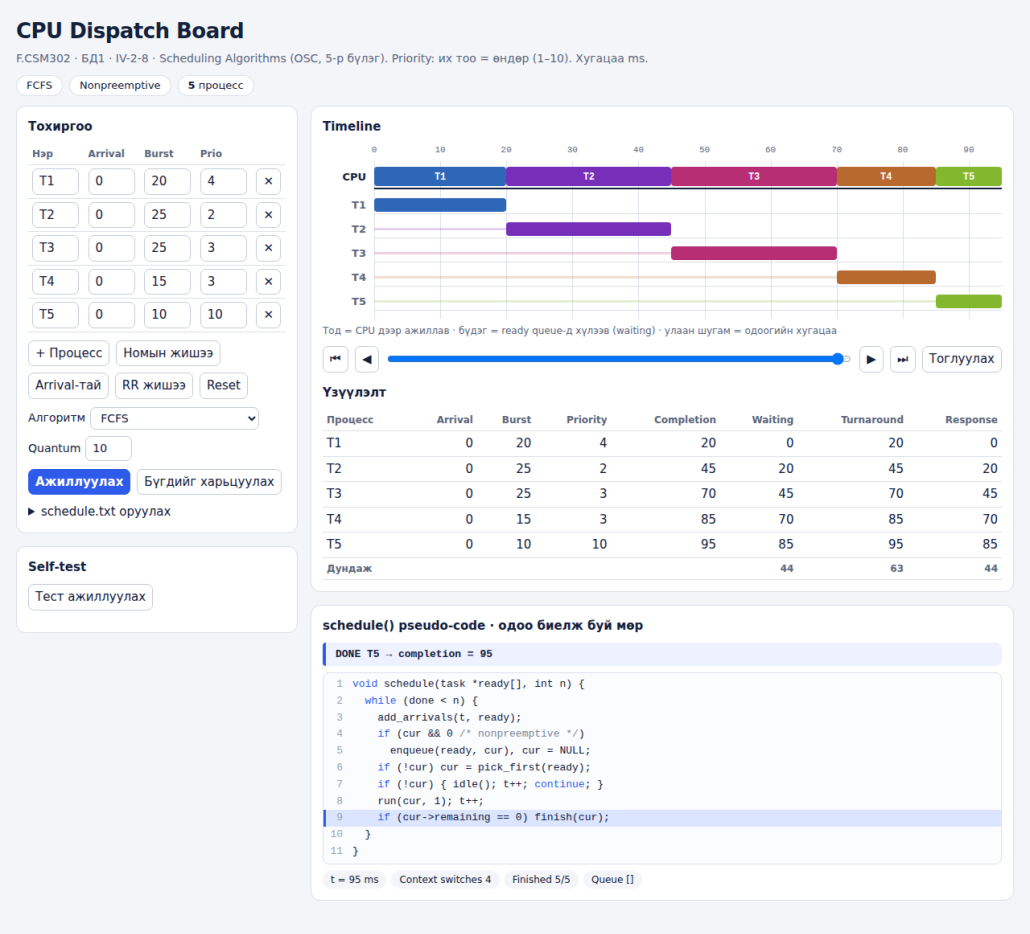
```
Running task = [T1] [4] [20] for 20 units.  (t = 0..20)
Running task = [T2] [2] [25] for 25 units.  (t = 20..45)
Running task = [T3] [3] [25] for 25 units.  (t = 45..70)
Running task = [T4] [3] [15] for 15 units.  (t = 70..85)
Running task = [T5] [10] [10] for 10 units.  (t = 85..95)

Task      tid  Pri  Burst   Waiting  Turnaround  Response
T1          1    4     20         0          20         0
T2          2    2     25        20          45        20
T3          3    3     25        45          70        45
T4          4    3     15        70          85        70
T5          5   10     10        85          95        85

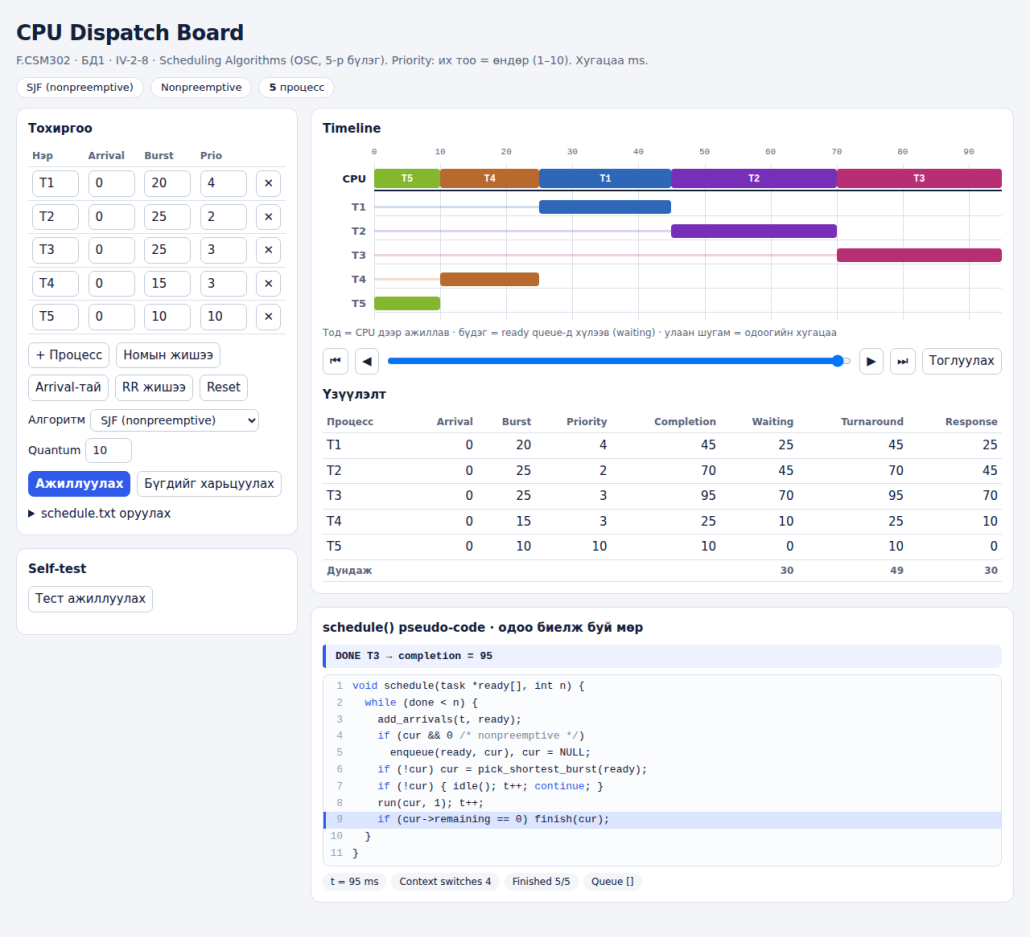
Average waiting time    = 44.00
Average turnaround time = 63.00
Average response time   = 44.00
```

Priority + RR (quantum 10), эхний 9 мөр — T3 ба T4 (priority 3) ээлжилж, T2 (priority 2) хамгийн сүүлд ажиллана:

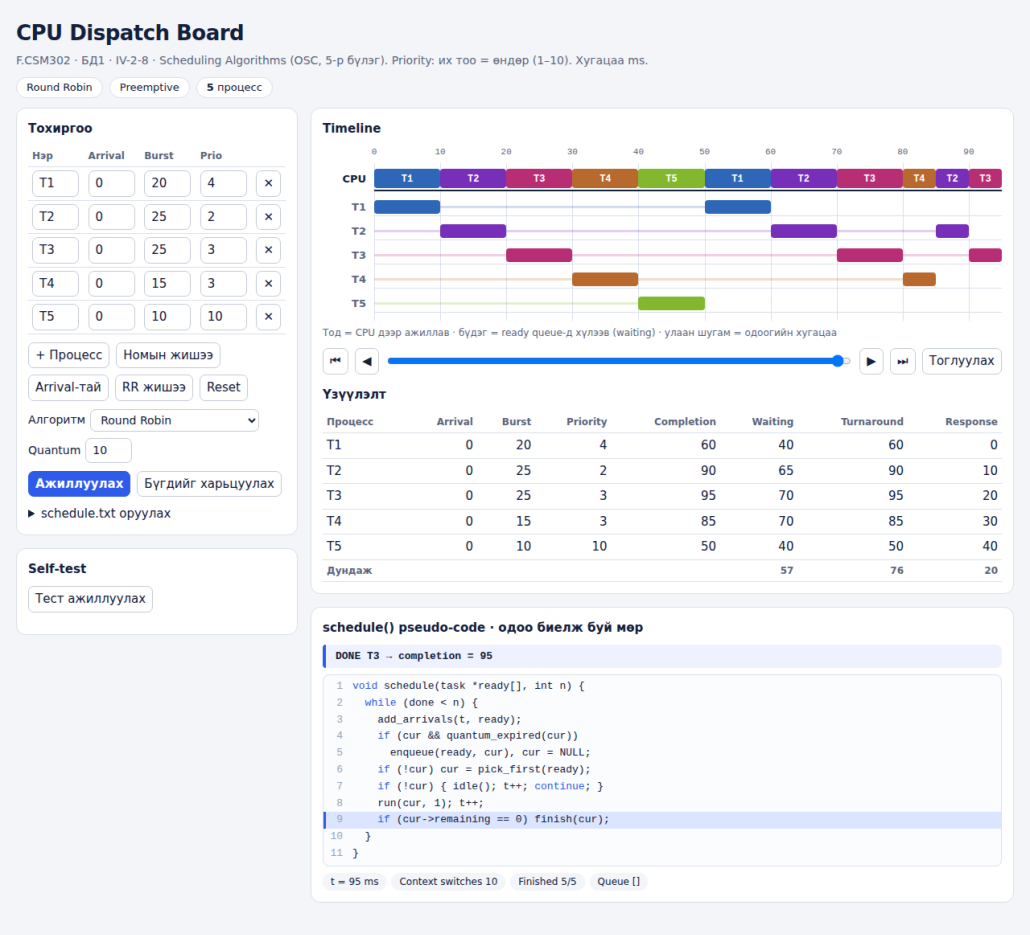
```
Running task = [T5] [10] [10] for 10 units.  (t = 0..10)
Running task = [T1] [4] [20] for 10 units.  (t = 10..20)
Running task = [T1] [4] [10] for 10 units.  (t = 20..30)
Running task = [T3] [3] [25] for 10 units.  (t = 30..40)
Running task = [T4] [3] [15] for 10 units.  (t = 40..50)
Running task = [T3] [3] [15] for 10 units.  (t = 50..60)
Running task = [T4] [3] [5] for 5 units.  (t = 60..65)
Running task = [T3] [3] [5] for 5 units.  (t = 65..70)
Running task = [T2] [2] [25] for 10 units.  (t = 70..80)
```



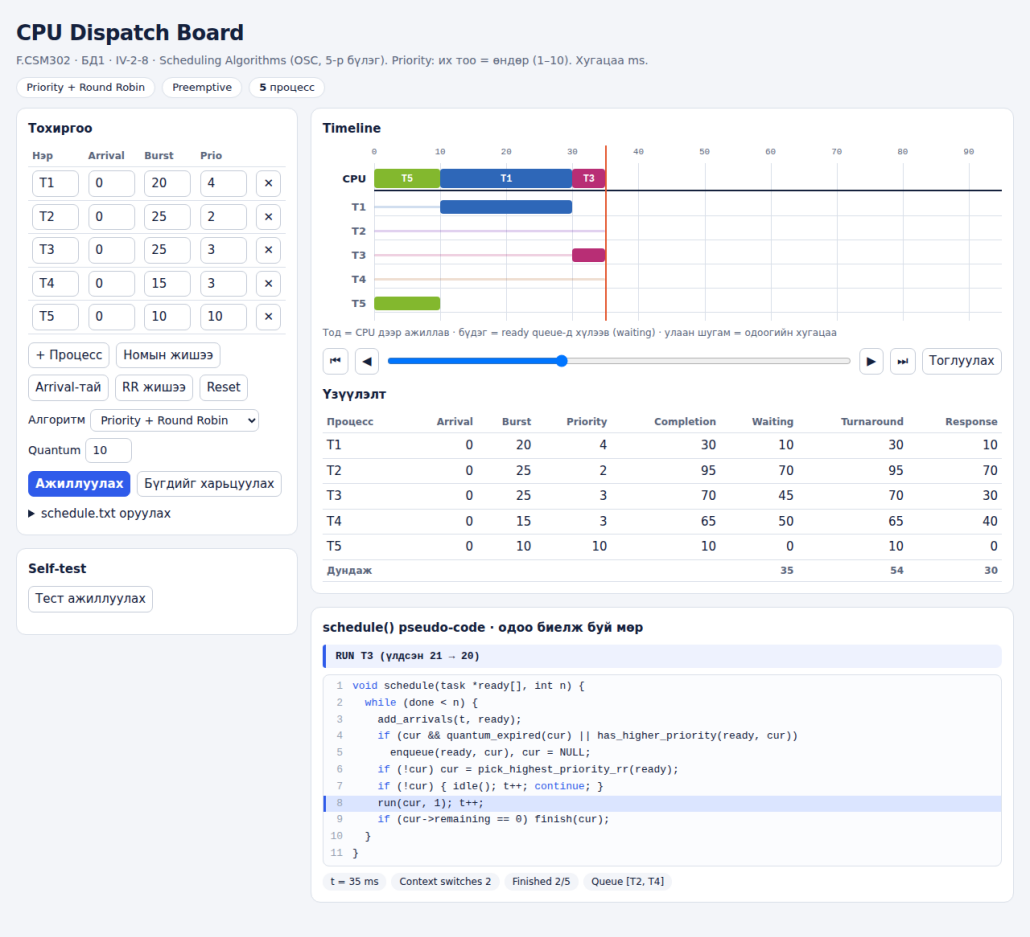
Зураг 2. FCFS — богино T5 нь хамгийн сүүлд (85 ms хүлээсэн).



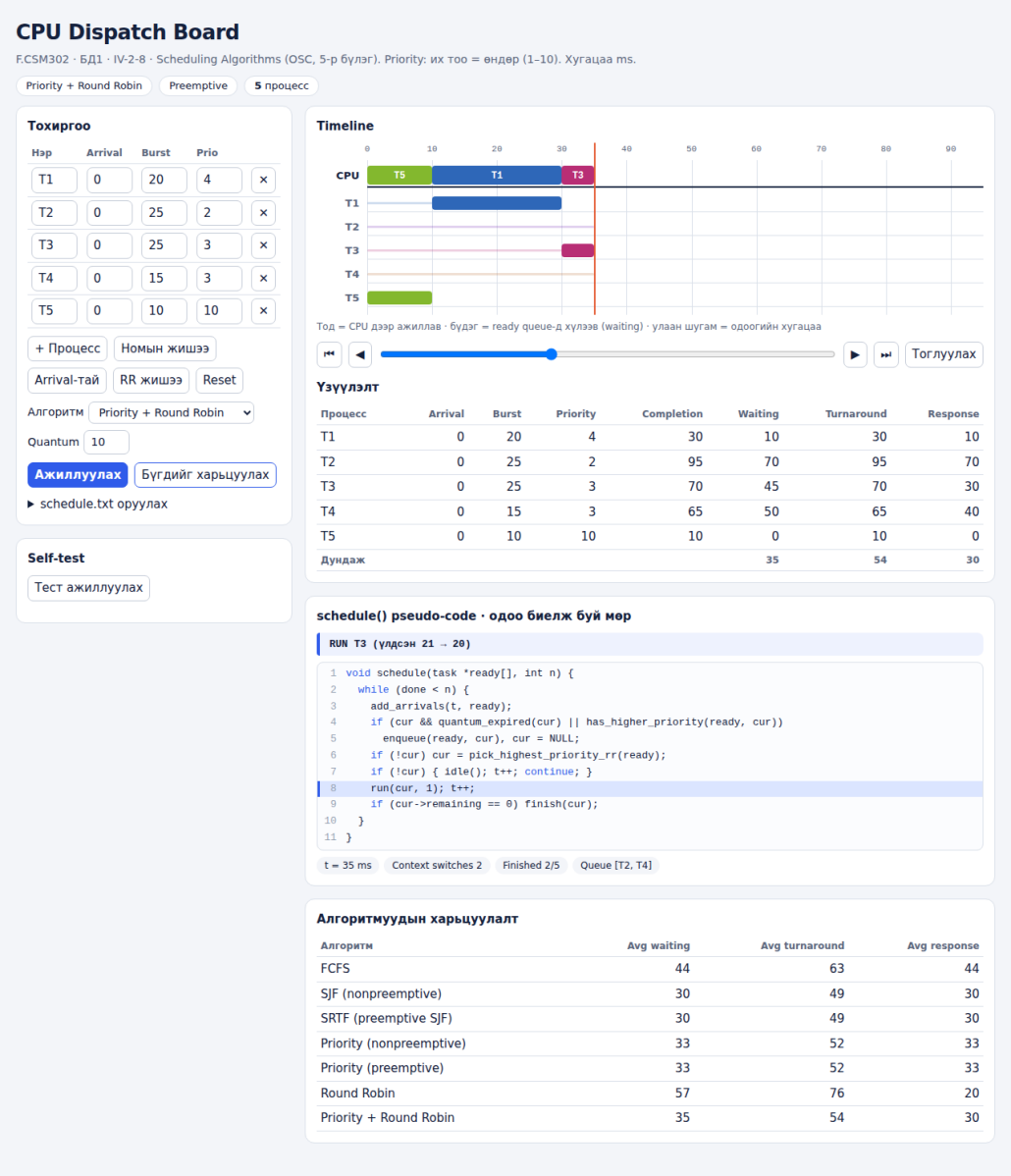
Зураг 3. SJF — богино task-ууд эхэлж, дундаж waiting хамгийн бага (30).



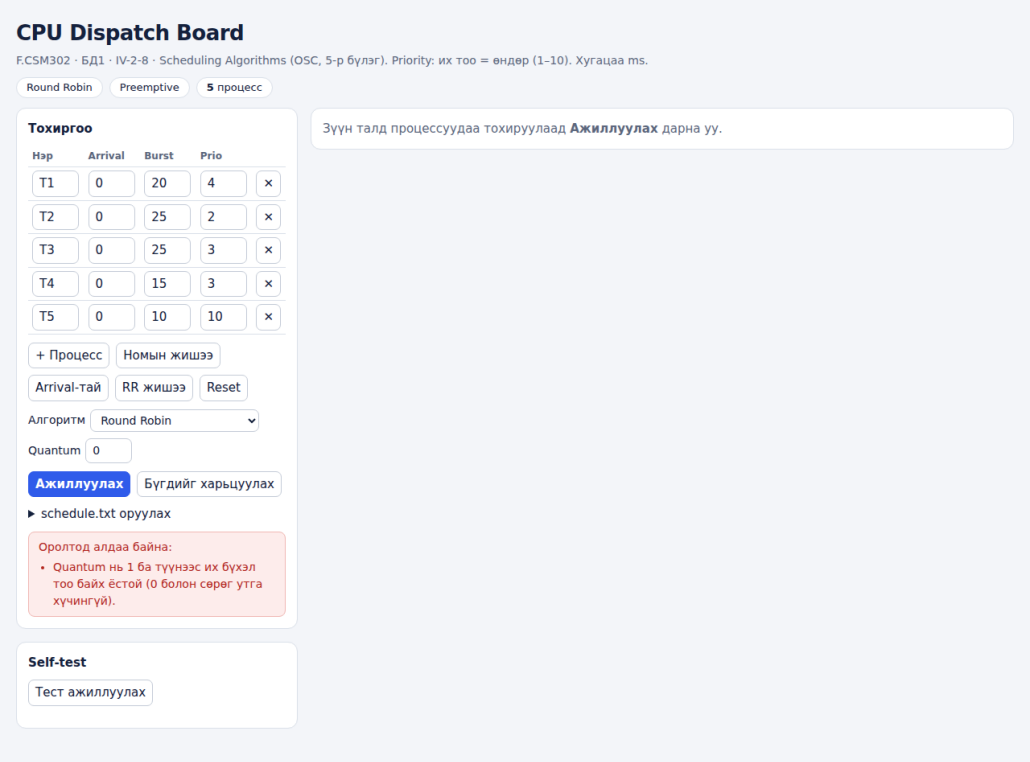
Зураг 4. Round-Robin (q = 10) — task бүр 10 ms-ээр ээлжилнэ; context switch 10.



Зураг 5. Priority + RR, t = 35 алхамд — T5, T1 дууссан; T3 ажиллаж байна; pseudo-code-ийн 8-р мөр тодорсон.



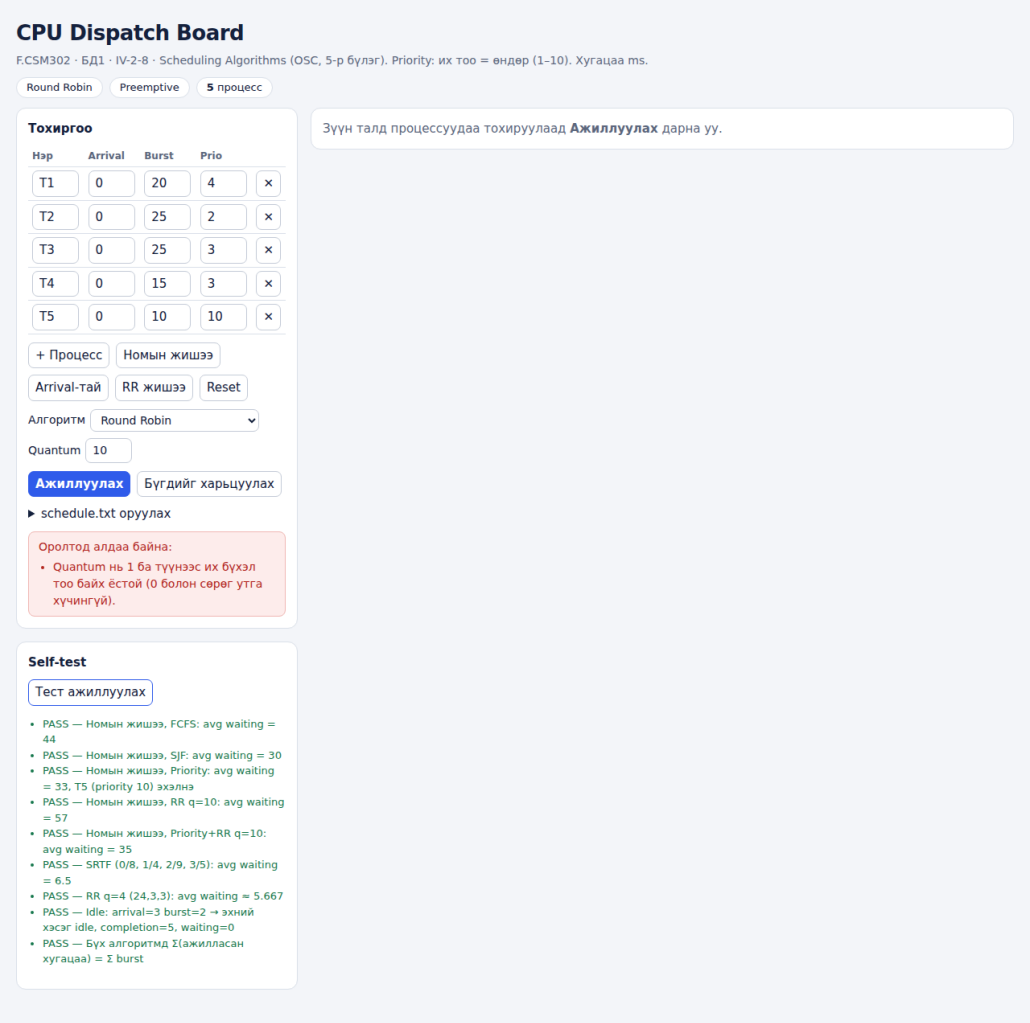
Зураг 6. "Бүгдийг харьцуулах" — 7 алгоритмын дундаж.



Зураг 7. Буруу оролт (quantum = 0) — ойлгомжтой алдааны мэдээлэл, ажиллуулалт хийгдэхгүй.

Алдаатай оролтод C хэрэгжилтийн хариу:

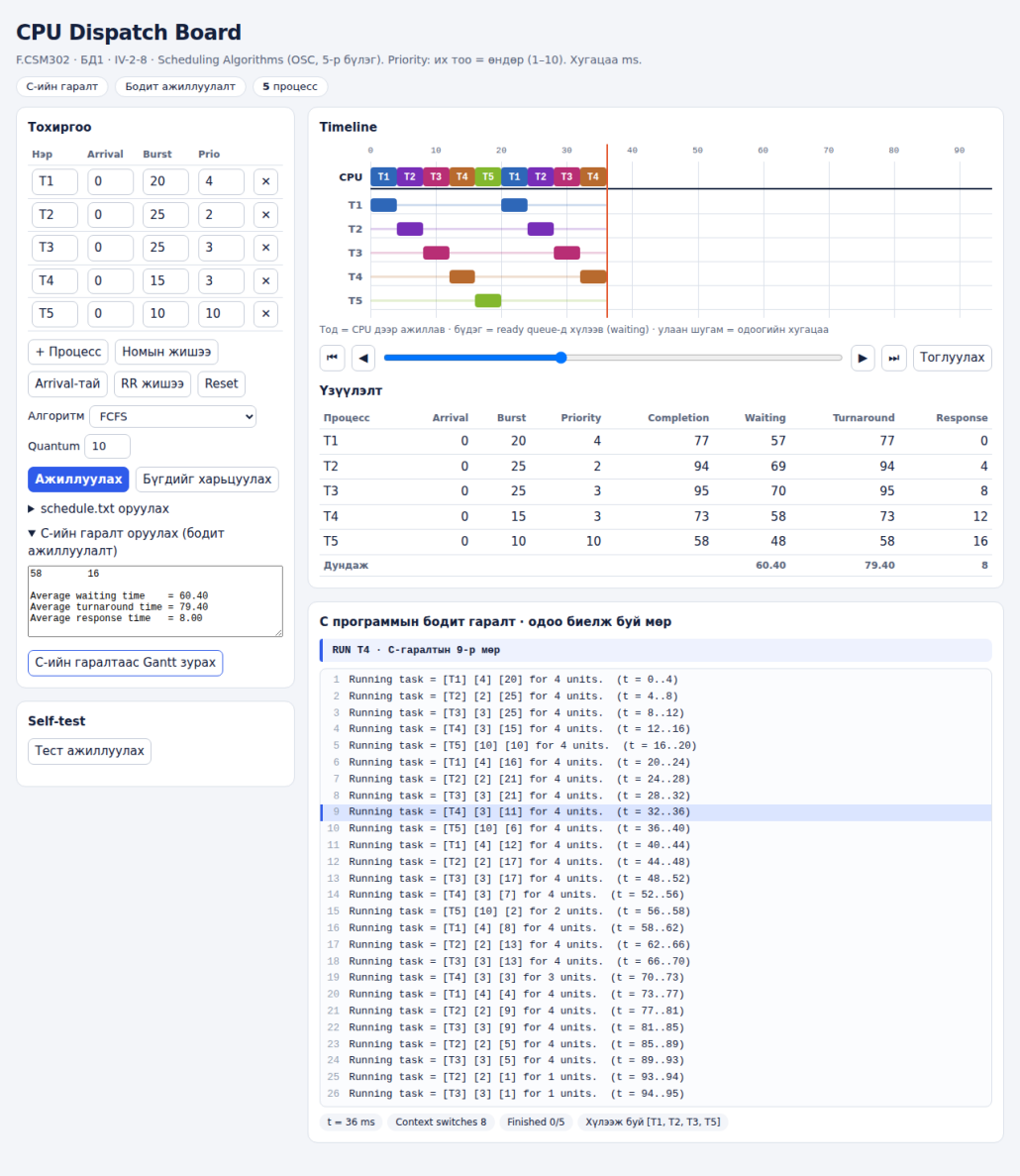
```
Error (../../tests/bad_priority.txt, line 2): priority 11 is outside 1..10.
Error: quantum must be an integer from 1 to 1000 (got "0").
```



Зураг 8. Симулятор доторх self-test — бүх шалгалт PASS.

UI ба C хэрэгжилтийн холбоо. C програмын 6 гаралтыг (FCFS, SJF, Priority, RR q=10, RR q=4, Priority+RR) UI-д буулгаж, UI-ийн тооцсон дундаж waiting / turnaround / response-ийг C-ийн өөрийн хэвлэсэн дундажтай автоматаар тулгасан — **6/6 таарсан**.

C гаралт	UI (waiting / turnaround / response)	C-ийн хэвлэсэн
FCFS	44 / 63 / 44	44.00 / 63.00 / 44.00
SJF	30 / 49 / 30	30.00 / 49.00 / 30.00
Priority	33 / 52 / 33	33.00 / 52.00 / 33.00
RR q=10	57 / 76 / 20	57.00 / 76.00 / 20.00
RR q=4	60.4 / 79.4 / 8	60.40 / 79.40 / 8.00
Priority + RR	35 / 54 / 30	35.00 / 54.00 / 30.00



Зураг 9. С-ийн бодит гаралтыг (RR, quantum 4) UI-д ачаалсан байдал, t = 36 ms: Gantt, метрик, С-гаралтын 9-р мөр тодорсон.

9. Хэлэлцүүлэг

- **SJF нь waiting time-ийг хамгийн бага** (30.00) болгосон нь онолтой нийцнэ. FCFS-д богино T5 (burst 10) 85 ms хүлээсэн нь convoy effect-ийн жишээ.
- **Priority** (33.00) нь SJF-д ойр боловч priority 2-той T2 хамгийн сүүлд (waiting 70) ажилласан нь starvation эрсдэлийг харуулна — өндөр priority-той task байнга ирвэл T2 хэзээ ч ажиллахгүй байж болно.
- **Round-Robin** нь response time хамгийн сайн (20.00) ч waiting (57.00) ба turnaround (76.00) хамгийн муу — богино task-ууд хүртэл олон удаа таслагдана. Quantum 4 болговол response 8 болж сайжирах боловч waiting 60.4 болж муудна (тест).
- **Priority + RR** (35.00) нь priority-ийн давуу талыг RR-ийн шударга байдалтай хослуулж, зөвхөн тэнцүү priority-той T3/T4 хооронд ээлжилсэн.
- **Хязгаарлалт:** (1) С хувилбар номын нөхцөлөөр бүх task t = 0 үед ирнэ гэж үзнэ; arrival, preemption-ийг зөвхөн симулятор дэмжинэ. (2) Context switch-ийн зардлыг тооцоогүй (0 ms). (3) Симулятор burst ≤ 1000, ≤ 20 task. (4) Симуляторын горимд Code Viewer нь pseudo-code; С-гаралтын горимд С-ийн гаралтын мөрүүдийг харуулна. (5) Симулятор нь

1 ms-ийн алхамтай, C нь slice-аар ажилладаг — үр дүн нь адил.

10. Дүгнэлт

Номын таван алгоритм ба хоёр нэмэлт даалгаврыг C хэл дээр бүрэн хэрэгжүүлж, *schedule.txt* оролт, *Makefile*-аар алгоритм сонгох номын загварыг дагасан. Gantt chart бүхий интерактив симулятор нь параметр өөрчилж дахин туршихад зориулагдсан бөгөөд C хэрэгжилттэй бие даасан хоёр хэрэгжилтийн үр дүн таарсан нь зөв ажиллагааны нотолгоо болсон. 18 автомат шалгалт давсан; буруу оролтыг ойлгомжтой мэдээлнэ. Үр дүн нь онолтой нийцсэн: SJF оновчтой waiting, RR хамгийн сайн response, Priority starvation-ийн эрсдэлтэй.

Ашигласан материал

- Silberschatz, Galvin, Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. 5-р бүлэг "CPU Scheduling", Programming Projects "Scheduling Algorithms" (P-29…P-31).
- GCC, GNU Make; Chromium, Playwright (screenshot автоматжуулалт).
- **AI ашигласан тухай:** Claude (Anthropic) кодын эхний хувилбар, тест, баримт бичгийн ноорог бичихэд ашиглагдсан. Демо видеоны хоолойг ElevenLabs (AI дуу хоолой)-оор үүсгэсэн; screenshot-ын нэг хэсгийг Playwright-аар автоматаар авсан. Багийнхан кодыг уншиж, ажиллуулж, үр дүнг шалгасан. **[багийн өөрийн хэсгийг нөхнө]**. Номын source code download-ийн файлыг ашиглаагүй.

Хавсралт А. Ажлын төлөвлөгөө

Талбар	Мэдээлэл
Хуваарилалтын код	IV-2-8
Төсөл	Scheduling Algorithms
Repository	**[GitHub холбоос]**
Хэл/сан/орчин	C (gcc, make); HTML/CSS/JS
Үндсэн дүрслэл	Gantt chart

Гишүүн	Хариуцах ажил	Commit/file
[Гишүүн 1]	C хэрэгжилт (driver, list, CPU, tasks)	**[commit]**
[Гишүүн 2]	Алгоритмууд (fcfs … priority_rr)	**[commit]**
[Гишүүн 3]	Симулятор UI (Gantt, алхамчилсан тоглуулалт)	**[commit]**
[Гишүүн 4]	Тест, expected.txt, run_tests.sh	**[commit]**
[Гишүүн 5]	Тайлан, Manual, пост	**[commit]**

Тест	Ороолт/нөхцөл	Хүлээгдсэн үр дүн
T1	tests/schedule.txt, FCFS	avg waiting 44.00
T2	tests/schedule.txt, SJF	avg waiting 30.00
T3	tests/schedule.txt, Priority	avg waiting 33.00
T4	tests/schedule.txt, RR q=10	avg waiting 57.00
T5	tests/schedule.txt, Priority+RR q=10	avg waiting 35.00
T6	schedule_ties.txt, Priority+RR q=4	avg waiting 12.75
T7	bad_priority.txt	алдааны мэдээлэл, exit ≠ 0
T8	rr … quantum 0	алдааны мэдээлэл, exit ≠ 0
T9	C-ийн 6 гаралтыг UI-д ачаалах	UI-ийн дундаж = C-ийн дундаж

Хавсралт В. Шаардлагын биелэлт

Шаардлага	Биелэлт
5 алгоритм	src/c/fcfs.c, sjf.c, priority.c, rr.c, priority_rr.c
Priority 1–10, quantum 10	task.h (MIN/MAX_PRIORITY, QUANTUM); driver.c-д шалгана
schedule.txt формат	driver.c — "name, priority, burst"; # тайлбар, хоосон мөр алгасна
driver / CPU.c run() / Makefile	driver.c, CPU.c, Makefile (make fcfs; ./fcfs schedule.txt)
Нэмэлт 1: атомар tid	tasks.c — __sync_fetch_and_add
Нэмэлт 2: дундаж метрик	tasks.c report()
Өөр өгөгдөл/параметр	schedule.txt, quantum аргумент; симулятор дээр бүх талбар
Буруу ороолт	driver.c шалгалт; симулятор readInput()
Gantt chart UI	src/web/index.html
UI ↔ C холбоо	src/web/index.html — "C-ийн гаралт оруулах" (parseC): C-ийн бодит гаралтаас Gantt зурна

Хавсралт С. GitHub дээрх ажиллагаа

Repository нь *IV-2-8_scheduler* нэг хавтас дотор **нэг** git repo байхаар зохион байгуулсан: README.md, src/, tests/, docs/, .gitignore. *.gitignore* нь хөрвүүлсэн executable-уудыг (fcfs, sjf, …) repo-д оруулдаггүй. Дараах хүснэгтийг багийнхан гүйцэтгэсний дараа бөглөнө.

Зүйл	Мэдээлэл
Repository URL	**[GitHub холбоос]**
Release / Tag	**[жишээ нь v1.0 — 9-р долоо хоног]**
Нийт commit	**[тоо]**
Branch / Pull request	**[ашигласан бол]**

Гишүүн	Commit-ын тоо	Гол commit / файл
[Гишүүн 1]	**[тоо]**	**[файл]**
[Гишүүн 2]	**[тоо]**	**[файл]**
[Гишүүн 3]	**[тоо]**	**[файл]**
[Гишүүн 4]	**[тоо]**	**[файл]**
[Гишүүн 5]	**[тоо]**	**[файл]**